

THREAT MODELING & CLOUD ARCHITECTURE PLAYBOOK

Design · Failure Domains · Control Selection · Measuring Effectiveness

Principal Product Security Engineer — Role-Specific Assessment Prep

The reasoning companion to the Hands-On Lab — built around the exact
Round-1 scenarios: JWT+OAuth threat models, VPC/KMS/tenant-isolation, NIST CSF.

HOW TO USE THIS PLAYBOOK

This is the design-and-reasoning half of your Round 2 prep, paired with the Hands-On Lab. Round 2 for a product-security role tests not just 'can you spot a bug' but 'can you threat-model a system, reason about failure domains, choose the right control at the right layer, and prove it works.' The Round 1 questions (cross-account KMS deletion, tenant isolation, VPC connectivity, NIST CSF mapping) signal these are fair game at depth.

How it's organized

- Part 1 — Threat-modeling walkthroughs: a repeatable five-step method, then STRIDE applied to the canonical JWT+OAuth+DB+compute system, with failure domains that shift between EC2 and serverless.
- Part 2 — AWS architecture scenarios: the exact Round-1 cloud topics answered precisely (VPC peering vs PrivateLink vs TGW, multi-AZ HA, cross-account KMS key deletion, K8s tenant isolation).
- Part 3 — Secure interaction, measurement, and governance: service-to-service security and failure domains, the four-lens method for MEASURING control effectiveness, NIST CSF 2.0 mapping, and DevSecOps/supply-chain/detection-engineering written-answer prep.

The answer shape graders reward

For any 'what controls would you put / how do you secure X' question, answer in this order: (1) decompose and name the trust boundaries, (2) enumerate threats (STRIDE), (3) state the failure domain / blast radius, (4) select controls at the correct layer, (5) say how you'd MEASURE that the control works. Steps 3 and 5 are what separate a senior answer from a principal one.

How this fits the full library

Hands-On Lab = find-the-vuln in code/IaC/K8s/policy. This Playbook = design, failure domains, control selection, measurement. Secure Coding Bank (170) = app-vuln depth. Advanced MCQ Bank (100) = reasoning drills. Together they cover both the hands-on and the explanatory halves of Round 2.

PART 1

Threat Modeling Walkthroughs

STRIDE applied to the JWT+OAuth+DB+compute scenario, failure domains by compute model, control selection + measurement

THE THREAT-MODELING METHOD (USE THIS EVERY TIME)

A principal is expected to produce a structured threat model on demand, not freestyle. Use this repeatable five-step method; it maps directly to the kind of scenario Round 1 hinted at (JWT+OAuth API, DB, session, EC2/serverless).

Five-step method

1. Decompose	Draw the data flow: components, data stores, and the trust boundaries data crosses. Note where authn/authz/encryption happen.
2. Enumerate (STRIDE)	Walk each element/flow through STRIDE: Spoofing, Tampering, Repudiation, Information disclosure, DoS, Elevation of privilege.
3. Identify failure domains	For each component, ask: if THIS fails or is compromised, what is the blast radius? What does the attacker reach next?
4. Select controls	For each top threat, choose a control and place it at the right layer. Prefer architectural/deterministic controls over advisory ones.
5. Measure	Define how you'd know the control works: a metric, a test, a detection. 'Untested control = assumed control.'

► STRIDE -> property -> typical control

Spoofing->Authentication (MFA, mTLS, signed tokens). Tampering->Integrity (signatures, HMAC, WORM). Repudiation->Non-repudiation (audit logs, signing). Information disclosure->Confidentiality (encryption, authz). DoS->Availability (rate limits, quotas, autoscale). Elevation->Authorization (least privilege, scoped roles).

SCENARIO 1 Threat-model a JWT + OAuth API: clients authenticate via OAuth, call an API that reads/writes a DB, with server-side sessions, running on EC2 (or serverless).

STRIDE walkthrough — the canonical Round-1 shape

Decompose the flow first: Client -> OAuth IdP (get token) -> API gateway/app (verify token, manage session) -> Database. Trust boundaries: Client<->IdP, Client<->API, API<->DB, API<->compute/cloud-control-plane. Then walk STRIDE across each element.

STRIDE for the JWT+OAuth API

S — Spoofing	Forged/replayed JWTs; stolen OAuth codes; impersonating the IdP. Controls: verify signature + pin alg, validate aud/iss/exp, PKCE + state, short token TTL, rotate session id at login.
T — Tampering	Modified claims (role user->admin), request body tampering, MITM. Controls: signed tokens (don't trust claims for authz; re-check server-side), TLS everywhere, integrity on stored data.
R — Repudiation	User denies an action; no audit trail. Controls: structured audit logs (authn, authz denials, privileged actions) in tamper-evident storage; correlation ids.
I — Info disclosure	Token/PII in URLs or logs; verbose errors; IDOR exposing other users' rows; secrets in stack traces. Controls: tokens in headers/cookies not URLs, generic errors, per-object authz, redact logs.
D — Denial of Service	Token-validation CPU abuse, expensive queries, login brute force, denial-of-wallet on serverless. Controls: rate limits/quotas, WAF, query timeouts, autoscale with caps, concurrency limits.
E — Elevation	alg:none / signature-not-verified -> forge admin; trusting client role claim; over-privileged compute IAM role. Controls: strict JWT validation, server-side authz from authoritative data, least-privilege instance/function role.

Controls

- Token layer: verify signature, pin algorithm, validate aud/iss/exp/sub; short-lived access tokens + rotating refresh; PKCE+state for the OAuth flow.
- Authorization: never trust token claims for entitlements — re-derive role/permissions from server state per request (defeats the user->admin escalation).
- Data layer: parameterized queries, per-object ownership checks (no IDOR), encryption at rest (KMS), TLS in transit.
- Session: rotate id at login (anti-fixation), HttpOnly/Secure/SameSite cookies, idle + absolute timeout.
- Compute: least-privilege EC2 instance role / Lambda execution role; IMDSv2; secrets from a manager, not env/source.

How to measure effectiveness

- % of API endpoints with automated authz tests (target 100% for object-level checks).
- JWT validation: a test suite asserting alg:none, wrong-aud, expired, and tampered tokens are all rejected.
- Detection: alert on auth failures spike, on tokens failing validation, on privilege-change events; measure mean-time-to-detect in tabletop.
- Coverage: % of compute with IMDSv2 enforced and least-privilege roles (CSPM check); secret-scanning coverage across repos.

◆ KEY — Key point

The exam's recurring trap is the user->admin JWT escalation. The two-part answer: (1) token INTEGRITY (verify signature, pin alg, validate claims) AND (2) don't use token claims as the authorization decision — re-check entitlements against server state.

SCENARIO 2 EC2-vs-serverless changes the failure domains. How does the threat model shift for a serverless (Lambda + API Gateway + DynamoDB) version of the same API?

failure domains by compute model

Same data flow, different blast radius and controls. Serverless removes host/patch concerns but shifts risk to IAM, event sources, and per-function configuration. The failure-domain question 'if this is compromised, what does the attacker reach' has different answers.

EC2 vs Serverless failure domains

Compromise of the compute	EC2: attacker gets a long-lived host -> steal instance-role creds (IMDS), pivot on the network, persist. Lambda: ephemeral; attacker gets the function's execution-role permissions for the invocation window, no persistent host.
Primary risk surface	EC2: OS/patching, network exposure, IMDS, host persistence. Serverless: over-broad function IAM role, event-source injection, dependency/layer supply chain, secrets in env vars.
Blast radius control	EC2: SG/NACL segmentation, IMDSv2, least-priv instance role. Serverless: one tightly-scoped role PER function, resource policies, VPC config for DB access.
DoS / cost	EC2: autoscaling caps. Serverless: reserved/maximum concurrency to bound denial-of-wallet; throttling at API Gateway.
Detection	EC2: host EDR + VPC flow logs + CloudTrail. Serverless: CloudTrail + Lambda logs + X-Ray; less host telemetry, more reliance on control-plane logs.

Controls

- Per-function least-privilege execution roles (not one shared role) — minimizes what a single compromised function can touch.
- Set reserved/maximum concurrency to bound cost and blast radius; throttle at API Gateway.
- Secrets from Secrets Manager/Parameter Store at runtime, never in environment variables in plaintext.
- Pin and scan Lambda layers/dependencies; sign deployment packages.
- For DB access, run the function in a VPC with SG-restricted access to the data store.

How to measure effectiveness

- % of functions with a dedicated (non-shared) least-privilege role; flag any function with wildcard actions.
- Concurrency caps configured on internet-facing functions (CSPM check).
- Mean time to patch a vulnerable dependency/layer across all functions.
- CloudTrail coverage and alerting on role assumption anomalies.

◆ KEY — Key point

Failure-domain framing graders reward: EC2 = persistent host, IMDS + network pivot + patching are the risks. Serverless = ephemeral, the IAM role IS the blast radius, so per-function least privilege and concurrency caps matter most.

SCENARIO 3 For the same system, walk the 'data at rest in a DB' element: what are the threats and the layered controls?*data store threat decomposition*

A data store is a high-value element: it concentrates confidentiality and integrity risk. Decompose threats by access path (app, admin, backup, replica, snapshot) rather than only the app path.

DB element — threats and controls

Confidentiality	Threats: SQLi/IDOR via app, direct network access, stolen snapshot/backup, over-broad DBA access. Controls: parameterized queries + per-object authz, private subnet + SG, encryption at rest (KMS) incl. snapshots, field-level encryption for the most sensitive data, least-privilege DB users.
------------------------	--

Integrity	Threats: unauthorized writes, tampered records, lost updates. Controls: least-priv write roles, optimistic locking/constraints, audit columns, immutable audit log, backups + PITR.
Availability	Threats: query DoS, deletion, ransomware. Controls: query timeouts, connection limits, automated backups with retention, deletion protection, multi-AZ.
Auditability	Threats: no record of access. Controls: DB audit logging of privileged/bulk access shipped to a separate account; alerting on anomalous reads.

Controls

- Network: private subnet, access only from the app SG (no public access).
- Crypto: KMS encryption at rest including automated snapshots; TLS in transit; consider app-layer field encryption for regulated fields.
- Access: distinct least-privilege DB users per service; no shared admin; IAM database auth to reduce standing credentials.
- Resilience: automated backups + point-in-time recovery, deletion protection, multi-AZ.

How to measure effectiveness

- % of data stores encrypted at rest with a CMK and not publicly accessible (CSPM).
- Backup restore test success rate and RTO/RPO measured, not assumed.
- Alerting on bulk-read / unusual-access patterns; measured detection time.
- Least-privilege: number of principals with write/admin to the store (drive toward minimum).

◆ KEY — Key point

Decompose a data store by ALL access paths (app, admin, backup, snapshot, replica), not just the app query path. Snapshots and backups are a commonly-missed exfiltration path — encrypt and access-control them too.

PART 2

AWS Cloud Architecture Scenarios

The exact Round-1 topics: VPC connectivity, multi-AZ HA, cross-account KMS deletion, K8s tenant isolation

SCENARIO 4 EC2 private communication within the same account but across different VPCs — what are the options and the secure choice?

AWS connectivity — VPC-to-VPC (Round-1 topic)

Two VPCs in the same account can't reach each other by default (separate address spaces). The question is how to connect them privately without traversing the internet, and at what scale.

VPC-to-VPC options

VPC Peering	Direct 1:1 private connection between two VPCs (same or cross-account/region). Non-transitive (A-B and B-C does NOT give A-C). Best for a small number of VPCs. No bandwidth bottleneck. Requires non-overlapping CIDRs.
Transit Gateway (TGW)	Hub-and-spoke router connecting many VPCs (and on-prem). Transitive routing, central control, scales to hundreds of VPCs. The right answer when you have many VPCs or need centralized routing/segmentation.
PrivateLink (VPC endpoint service)	Exposes a SPECIFIC service (not the whole VPC) privately via an interface endpoint. Best when one VPC only needs to consume a single service in another VPC, with no network-level reachability between the VPCs.

Controls

- Few VPCs, full network reachability needed -> VPC Peering with tight route tables and SGs.
- Many VPCs / central segmentation / on-prem -> Transit Gateway with route-table segmentation per environment.
- Only one service needs exposure, want least network exposure -> PrivateLink (most least-privilege option).
- In all cases: non-overlapping CIDRs, security groups restricting to required ports, and flow logs.

How to measure effectiveness

- No public path exists (routes verified); flow logs confirm traffic stays private.
- Segmentation: route tables only permit intended VPC pairs; periodic review of peering/TGW attachments.

◆ KEY — Key point

Decision rule: 2-3 VPCs -> Peering; many VPCs -> Transit Gateway (transitive); single-service exposure with minimal reachability -> PrivateLink. PrivateLink is the most least-privilege because it exposes a service, not a network.

SCENARIO 5 Highly available private connection between Availability Zones within a single account/VPC.

AWS HA across AZs (Round-1 topic)

Within one VPC, subnets are AZ-scoped, but the VPC itself spans AZs and inter-AZ traffic is already private over the AWS backbone. HA comes from deploying across multiple AZs and fronting them with AZ-aware, redundant components — not from a special 'link'.

Multi-AZ HA building blocks

Subnets per AZ	Create subnets in ≥ 2 (ideally 3) AZs; deploy workloads across them. Inter-AZ traffic stays on the private AWS network.
Load balancing	An ALB/NLB spanning the AZs distributes traffic and routes around a failed AZ; enable cross-zone load balancing.
Data tier	RDS Multi-AZ (synchronous standby in another AZ, automatic failover) or Aurora (storage replicated across 3 AZs). DynamoDB/S3 are inherently multi-AZ.
Stateless compute	Auto Scaling group across AZs so capacity is replaced automatically; no single-AZ dependency.

Redundant NAT	One NAT gateway per AZ (not a single shared NAT) so egress survives an AZ failure.
----------------------	--

Controls

- Span ≥ 2 AZs for every tier; use AZ-aware managed services (Multi-AZ RDS, cross-zone LB).
- Per-AZ NAT gateways and per-AZ subnets to avoid a single-AZ dependency in the egress/data path.
- Keep all inter-AZ traffic within the VPC/backbone (private) — no internet hop.
- Security unchanged by HA: SG/NACL rules, encryption, and flow logs still apply per subnet.

How to measure effectiveness

- Architecture review confirms no tier is single-AZ; chaos/failover test that killing one AZ keeps the service up (measured RTO).
- RDS failover tested; measured failover time vs RTO target.

◆ KEY — Key point

There is no special private 'inter-AZ link' to buy: inter-AZ traffic in a VPC is already private. HA = deploy across multiple AZs with AZ-aware redundant components (multi-AZ data, cross-zone LB, per-AZ NAT, ASG). Avoid single-AZ chokepoints.

SCENARIO 6 Cross-account KMS CMK: account B uses a key owned by account A, and a client in account B wants to delete (schedule deletion of) the key. Is that possible, and how should it be controlled?

AWS KMS cross-account (Round-1 topic)

Deletion of a KMS key is governed by the key's RESOURCE policy (and the owning account's IAM), not by the consuming account. Whether account B can schedule deletion depends entirely on what account A's key policy granted. The secure design grants consumers only usage, never administration.

Who can do what to a cross-account CMK

Key ownership	The CMK lives in account A. Account A's key policy is the root of trust; nothing happens to the key that A's policy doesn't allow.
What B should get	Only kms:Encrypt, kms:Decrypt, kms:GenerateDataKey, kms:DescribeKey — usage, scoped to B's principals.
What B should NOT get	kms:ScheduleKeyDeletion, kms:DisableKey, kms:PutKeyPolicy, kms:Create*/Update*/Revoke* — administration stays with A's key admins only.
If B can delete it	That means A's key policy over-granted admin actions to B. A should remove those; key deletion must require A's key-admin principals.
Blast radius of deletion	Scheduling deletion (min 7, set to 30 days) makes ALL data encrypted under the key unrecoverable after the window. Hence deletion is the most dangerous KMS action.

Controls

- Separate key administration from key usage in the key policy (see Lab 1, KMS drill). Consumers get usage actions only.
- Grant the cross-account principal ONLY Encrypt/Decrypt/GenerateDataKey/DescribeKey; keep ScheduleKeyDeletion with account A's key admins.
- Set deletion_window to 30 days, enable rotation, and alert on ScheduleKeyDeletion via CloudTrail/EventBridge so an erroneous or malicious deletion can be cancelled.
- Consider a key policy condition (e.g. kms:ViaService) and SCP/permission boundaries to further constrain.

How to measure effectiveness

- Key-policy review: no cross-account principal holds Schedule/Disable/PutKeyPolicy.
- Alerting in place on ScheduleKeyDeletion with a runbook to CancelKeyDeletion within the window; tested.

- Rotation enabled; deletion window = 30 days on all customer-data CMKs.

◆ KEY — Key point

Crisp answer: a consumer account can only delete a cross-account key if the OWNER's key policy granted it kms:ScheduleKeyDeletion — which it never should. Grant consumers usage-only; keep deletion/admin with the owner; alert on deletion attempts; 30-day window for recovery.

SCENARIO 7 Tenant A's pod can access Tenant B's secrets in their namespace, even though IAM is perfectly configured. Why, and how do you fix it?

Kubernetes multi-tenant isolation (Round-1 topic)

This is a control-layer mismatch. Cloud IAM governs access to CLOUD APIs (S3, KMS, etc.). It does NOT govern access to Kubernetes Secrets inside the cluster — Kubernetes RBAC and namespace boundaries do. 'Perfect IAM' is irrelevant to in-cluster secret access.

Why IAM doesn't isolate K8s secrets

What IAM controls	Calls to AWS APIs from pods (via IRSA/the node role). E.g. which S3 bucket or KMS key a pod can use.
What governs K8s Secrets	Kubernetes RBAC (Roles/RoleBindings) scoped to namespaces. A pod's service account with secrets:get in a namespace can read those Secrets regardless of IAM.
The failure	If tenants share a namespace, or a Role grants secrets access too broadly, a pod reads other tenants' Secrets — IAM never enters the picture.
etcd dimension	K8s Secrets are base64 in etcd by default. Without encryption-at-rest for etcd, node/etcd access also exposes them.

Controls

- Namespace-per-tenant: the namespace is the RBAC/NetworkPolicy/quota boundary. Never co-tenant in one namespace.
- Least-privilege RBAC: each tenant's service accounts can read only their own namespace's specific Secrets (resourceNames).
- Default-deny NetworkPolicy between namespaces; automountServiceAccountToken:false where the API isn't needed.
- Encrypt etcd at rest (EncryptionConfiguration / KMS provider); prefer an external secret store (External Secrets Operator -> per-tenant Secrets Manager paths).
- Enforce the structure with admission policy (Kyverno/Gatekeeper): no cross-namespace secret refs, namespace-per-tenant required.

How to measure effectiveness

- RBAC review: no service account can read Secrets outside its tenant namespace (automated check).
- Admission policy in Enforce mode blocking violations; audit logs show zero cross-namespace secret access.
- etcd encryption enabled; % of tenants on the namespace-per-tenant model.

◆ KEY — Key point

The graded insight: identify the CORRECT control layer. Kubernetes Secret access = RBAC + namespace boundaries, NOT cloud IAM. Multi-tenant isolation needs namespace-per-tenant, least-privilege RBAC, NetworkPolicy, and etcd encryption — IAM correctness doesn't help here.

SCENARIO 8 How do you decide between VPC Peering, PrivateLink, and Transit Gateway for connecting services? Give the reasoning, not just the answer.

connectivity decision framework

The decision turns on three axes: how many things connect, whether you need full network reachability or just one service, and whether you need transitive routing or central control. Pick the most least-privilege option that meets the need.

Connectivity decision matrix	
Need: a few VPCs, full reachability	VPC Peering. Simple, no bandwidth limit, but non-transitive and N:N peering doesn't scale (mesh explosion).
Need: many VPCs / on-prem / central control	Transit Gateway. Transitive hub-and-spoke; segment with TGW route tables per environment.
Need: expose ONE service, minimal reachability	PrivateLink. Consumer reaches only the published service via an interface endpoint; no route between the VPCs. Most least-privilege.
Need: reach an AWS service privately	Gateway endpoint (S3/DynamoDB) or interface endpoint (most others). Keeps service traffic off the internet.

Controls

- Default to the option that exposes the least: PrivateLink (service) < Peering (two VPCs) < TGW (many VPCs).
- Whatever you choose: non-overlapping CIDRs, SG/route-table least privilege, flow logs, and endpoint policies where supported.

How to measure effectiveness

- Attachment/route review confirms only intended connectivity exists; no transitive path leaks.
- Flow logs validate traffic stays private and matches intended flows.

◆ KEY — Key point

Reasoning graders want: least exposure first. PrivateLink exposes a single service (best isolation); Peering connects two whole VPCs; Transit Gateway connects many with transitivity (most reach, needs segmentation discipline).

PART 3

Secure Interaction, Measurement & Governance

Service-to-service security, failure domains, measuring control effectiveness, NIST CSF mapping, DevSecOps & supply chain

SCENARIO 9 How should two internal services communicate securely? Walk the controls and their failure domains.

service-to-service interaction

Design service-to-service trust as zero-trust: the network gives no implicit trust. Each call must be authenticated, authorized, encrypted, and auditable, with controls placed so that the failure of one is caught by another.

Controls for service-to-service calls

Authentication	mTLS (SPIFFE/SPIRE identities) or signed service tokens (OAuth client-credentials). Each service proves its identity; network position confers nothing.
Authorization	The callee authorizes the caller's identity per operation (not just 'is on the network'). Central policy (OPA/Cedar) or per-service checks.
Encryption	TLS/mTLS in transit for every hop, including inside the cluster/mesh.
Identity propagation	Propagate the END-USER identity in a SIGNED, verified token (not a mutable header) so the callee can make per-user decisions (avoids confused-deputy).
Resilience	Timeouts, retries with backoff, circuit breakers, and rate limits between services so one slow/failing service can't cascade.
Audit	Log every inter-service call (caller, callee, operation, decision) for detection and forensics.

Controls

- mTLS for mutual authentication + encryption (service mesh makes this default).
- Per-operation authorization on the callee using the caller's verified identity.
- End-user context propagated as a signed token; callee re-verifies — never trust an 'X-User-*' header.
- Timeouts/circuit breakers/bulkheads to contain failure; rate limits to contain abuse.

How to measure effectiveness

- % of internal traffic on mTLS (mesh telemetry); target 100%.
- Authz coverage: every inter-service endpoint has an enforced policy (tested).
- Resilience tested: fault injection shows a failing dependency doesn't cascade (measured).

◆ KEY — Key point

Failure-domain framing: if service A is compromised, mTLS + per-operation authz limit it to exactly the calls A is allowed to make; signed user-context prevents it from acting as arbitrary users; circuit breakers stop a failure becoming an outage. Network reachability alone is never authorization.

SCENARIO 10 A payment service calls a third-party API and writes to a queue consumed by a fulfillment service. Identify the trust boundaries and the controls at each.

multi-component interaction + failure domains

Map each boundary crossing and place a control at each. Treat the queue and the third party as untrusted-input sources to their consumers.

Boundaries and controls

Client -> Payment svc	AuthN (user token), input validation, idempotency key (avoid double charge), rate limit.
Payment svc -> 3rd-party API	Outbound auth (API key from secret manager), TLS + cert validation, timeout + circuit breaker, egress allowlist (SSRF/exfil control), retry with idempotency.

Payment svc -> Queue	Least-privilege producer IAM (send-only), message integrity (sign/verify if cross-trust), encryption at rest (KMS).
Queue -> Fulfillment svc	Consumer treats messages as UNTRUSTED input: validate schema, authorize the implied action, idempotent processing (messages can be redelivered), poison-message/DLQ handling.

Controls

- Idempotency keys end-to-end so retries/redeliveries don't double-charge or double-fulfill.
- Egress allowlist + timeout + circuit breaker on the third-party call (contains SSRF, exfil, and cascading failure).
- Least-privilege queue IAM: producer can only send, consumer can only receive on its queue; encryption at rest.
- Consumer-side schema validation and authorization — never trust a message just because it's on your queue.

How to measure effectiveness

- Duplicate-side-effect rate (double charges/fulfillments) ~ 0 under retry/redelivery tests.
- DLQ depth and poison-message handling monitored; alert on growth.
- Egress allowlist coverage; third-party call timeout/breaker tested via fault injection.

◆ KEY — Key point

A queue is a trust boundary: the consumer must validate and authorize messages as untrusted input, and process idempotently (at-least-once delivery means redelivery WILL happen). Egress controls on third-party calls double as SSRF/exfil and resilience controls.

MEASURING CONTROL EFFECTIVENESS

A principal is expected to answer 'how do you know the control works?' — not just name controls. Tie each control to a measurable signal across four lenses: coverage, strength, detection, and outcome.

Four lenses for measuring a control

Coverage	What fraction of the estate has it? E.g. % of buckets with Block Public Access, % services on mTLS, % repos with SAST/secret-scanning, % compute on IMDSv2. Gaps are where breaches happen.
Strength / correctness	Does it actually stop the attack? Test it: a JWT suite that proves alg:none/expired/wrong-aud are rejected; a pen test that the IDOR is fixed; policy unit tests (confest/kyverno test) proving the rule denies the bad case.
Detection	If the control is bypassed, would you know? Mean time to detect (MTTD) in tabletop/purple-team exercises; alert fidelity (true-positive rate); % of attack techniques with a detection (ATT&CK coverage).
Outcome	Does it reduce risk over time? MTTR by severity, findings open past SLA, recurrence rate of a vuln class, reduction in exposed surface quarter over quarter.

▶ The principle to state

'An untested control is an assumed control.' Effectiveness = coverage x strength x detectability, validated by testing, and trended via outcome metrics (MTTR, recurrence). Always pair a control with the metric/test that proves it works.

Control -> how to measure it (examples)

WAF / input validation	% requests inspected; bypass-test results; false-positive rate; blocked-attack trend.
-------------------------------	---

IAM least privilege	# principals with wildcard/admin (drive down); unused-permission reports (Access Analyzer); PassRole scoping coverage.
Admission policy (Kyverno/OPA)	% clusters with policy in Enforce; policy unit tests pass; count of violations blocked; drift between policy and running workloads.
Secret management	Secrets-in-code detections; mean time to rotate after exposure; % secrets in a manager vs env/source.
Patch / SCA	% fleet on latest critical patch; MTTR for KEV-listed CVEs; # high/critical deps open past SLA.
Detection engineering	ATT&CK technique coverage; MTTD in exercises; alert true-positive rate; % alerts with a runbook.

NIST CSF 2.0 -> CLOUD-NATIVE CONTROL MAPPING

Explanatory questions often ask you to map cloud-native controls to a framework. NIST CSF 2.0 has six functions: Govern, Identify, Protect, Detect, Respond, Recover. Map your controls to each — it shows breadth and structure.

CSF function -> cloud-native controls

GOVERN	Cloud security policy, ownership, risk tolerance, SCPs/Org policy, control standards (CIS benchmarks), policy-as-code governance (OPA/Kyverno baselines).
IDENTIFY	Asset inventory (CSPM, Config), data classification, SBOMs, threat modeling, attack-surface mapping, dependency inventory.
PROTECT	IAM least privilege, encryption (KMS, TLS), network segmentation (SG/NACL/NetworkPolicy), secrets management, admission control, Pod Security, image signing, hardening baselines.
DETECT	CloudTrail/flow logs, GuardDuty, Falco/eBPF runtime, SIEM detections, anomaly detection, drift detection, container runtime scanning.
RESPOND	IR runbooks, automated containment (isolate instance/revoke creds), SOAR playbooks, on-call/escalation, forensics readiness.
RECOVER	Backups + tested restore, multi-AZ/region DR, immutable backups (ransomware), post-incident review and control improvement.

▶ How to use this in an answer

When asked to design or assess a cloud security program, structure the answer by CSF function. It guarantees you cover prevention AND detection AND response/recovery — a common gap is naming only PROTECT controls and forgetting DETECT/RESPOND/RECOVER.

DEVSECOPS, SUPPLY CHAIN & DETECTION ENGINEERING (WRITTEN-ANSWER PREP)

Shifting left (and right)

- Shift left = integrate security early: threat modeling at design, IDE/pre-commit secret scanning, SAST/SCA/IaC scan in CI, policy-as-code tested before merge. Cheaper to fix earlier.
- Shift right (complement) = runtime protection and detection in production: admission control, runtime threat detection (Falco), CSPM drift, continuous re-scan for new CVEs. Left and right together = defense in depth across the lifecycle.
- Gates, not just scans: each tool has an explicit fail condition (e.g. 'fail on HIGH/CRITICAL + KEV + secrets'); block actionable issues, report-only on noise so teams don't bypass CI.

Software supply chain security

- Threats: malicious/compromised dependencies (event-stream, ua-parser-js, XZ), dependency confusion, typosquatting, compromised build pipeline (SolarWinds), unsigned artifacts.
- Controls: pin versions+hashes, scope internal packages to private registries, SCA in CI, SBOM (syft/CycloneDX), sign artifacts (cosign/Sigstore), build provenance (SLSA/in-toto), verify signatures at admission, reproducible builds, least-privilege CI with OIDC (no long-lived keys).
- Frameworks to name: SLSA (build provenance levels), in-toto (attestations), Sigstore/cosign (signing), CycloneDX/SPDX (SBOM formats).

Detection engineering and scaling it

- Detection engineering = treating detections as code: write, version, test, and tune detections (Sigma rules, SIEM queries) mapped to MITRE ATT&CK techniques.
- Scaling approach: standardize log pipelines (structured logs, central SIEM/data lake); detection-as-code in git with CI tests; coverage measured by ATT&CK technique; reduce false positives via tuning and enrichment; automate triage/response (SOAR) for high-confidence detections.
- Measure: ATT&CK coverage %, MTTD in purple-team exercises, alert true-positive rate, % alerts with a runbook, detection-as-code test pass rate.

Cloud-native controls (one-liners to recall)

- Code: SAST, secret scanning, secure coding standards, dependency pinning.
- IaC: checkov/tfsec/kics in CI; policy-as-code; no hardcoded secrets; least-privilege modules.
- Secrets: KMS/Secrets Manager/Vault; OIDC for CI; rotation; no secrets in images/env/source.
- Containers: minimal/distroless base, scan (trivy/grype), SBOM, sign (cosign), non-root, drop caps, read-only fs.
- Kubernetes: Pod Security Admission (restricted), NetworkPolicy default-deny, least-priv RBAC, admission policy (Kyverno/OPA), etcd encryption.
- Runtime: Falco/eBPF behavioral detection, CSPM drift, GuardDuty, image re-scan for new CVEs.

▶ Answer structure for explanatory questions

State the threat, name the controls mapped to lifecycle stage (or CSF function), specify the ENFORCEMENT point and gate, and finish with how you'd MEASURE effectiveness. That four-part shape (threat -> controls -> enforcement -> measurement) is what reads as principal-level.

